Radius

The Confidence Layer for Modern Software Development

Final Year Project - Introduction Round Brief

This document explains the idea, technical workflow, current proof of concept, completed work, and roadmap for Radius. It is written for professors, teammates, and reviewers who need to understand the project clearly without reading the entire codebase.

Project	Radius - AI-native software verification platform
Core question	Is this pull request safe to merge?
Current phase	Phase 1 deterministic repository intelligence and verification foundation
Primary users	Developers, technical leads, DevOps/platform teams, and organizations using AI-assisted development
Prepared from	Repository docs and code status on 22 Aug 2026

1. Executive Summary

Radius is a software verification platform that analyzes code changes before merge and produces explainable evidence about risk and confidence. It does not replace GitHub, CI/CD, or existing test frameworks. Instead, it sits between code changes and deployment, understands the repository structure, computes the affected area of a change, selects relevant validations, and presents a confidence report.

In simple terms:

Radius helps a reviewer move from only asking 'Did the tests pass?' to asking 'What changed, what can break, which tests matter, and how confident are we before merging?'

Why this matters

- AI coding tools and fast development cycles can create large changes quickly.
- A passing test suite may still miss hidden dependencies or untested impact areas.
- Reviewers need explainable evidence, not a black-box score.
- Radius focuses first on deterministic analysis, so results are traceable and repeatable.

Project positioning

- Radius is a repository intelligence and pull request verification system.
- Radius is not a CI/CD replacement, source hosting platform, formatter, or AI coding assistant.
- Future AI features will enhance summarization and recommendations, but Phase 1 decisions are deterministic.

2. Problem and Proposed Solution

Problem

Traditional pipelines mostly report whether existing tests passed. They usually do not explain the full impact of a change, which modules depend on modified code, whether selected tests are sufficient, or what evidence supports merge readiness.

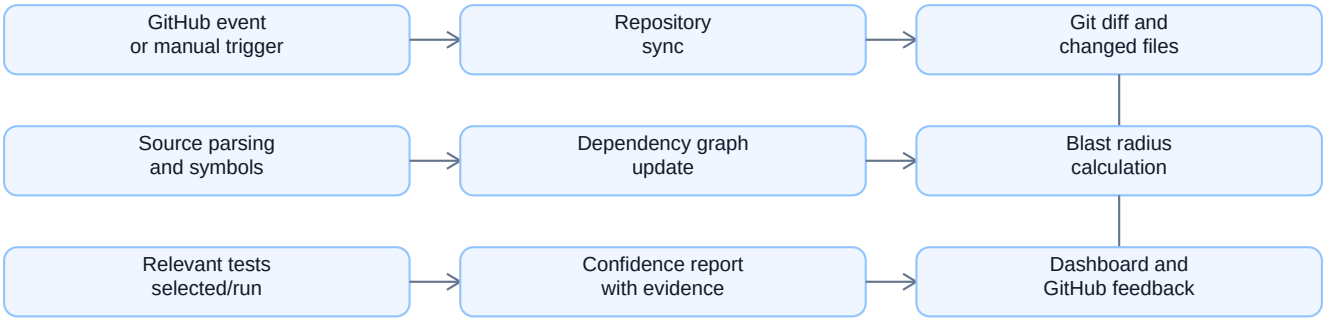
Current gap	Why it is a problem	Radius response
Only pass/fail test output	Reviewers cannot see what remains unverified.	Attach confidence evidence and recommendations.
No repository-level understanding	Dependency impact is hard to inspect manually.	Parse code and build dependency graphs.
Large pull requests are risky	Hidden affected files and modules can be missed.	Compute blast radius from graph traversal.
Tests may be too broad or too narrow	Teams waste time or miss targeted validation.	Select relevant tests based on impacted paths.

Solution concept

Radius builds a deterministic verification pipeline. A GitHub event or manual trigger starts an analysis. Radius synchronizes repository context, reads the diff, parses source files, updates a dependency graph, computes affected files/modules/tests, executes relevant validations, and produces an explainable confidence report visible in the dashboard and, in the target flow, GitHub feedback.

3. End-to-End Workflow

The core workflow is intentionally pipeline-based. Each step receives structured input, produces structured output, and remains loosely coupled from the next step.



Workflow details

Step	What Radius does	Output
1. Trigger	Receives GitHub webhook or manual analysis request.	Analysis record/job
2. Repository	Clones/fetches/checks out repository context.	Local repository snapshot
3. Diff	Reads Git changes and normalizes added/modified/deleted/renamed files.	Changed-file evidence
4. Parser	Extracts imports, exports, classes, functions, methods, and symbols.	Repository metadata
5. Graph	Builds file/module/symbol dependency relationships.	Graph snapshot
6. Blast radius	Traverses reverse dependencies from changed files.	Affected files, modules, symbols, tests
7. Testing	Discovers and plans/runs relevant existing tests.	Test selection and run result
8. Confidence	Scores risk from deterministic evidence.	Confidence score, risk level, recommendations
9. Presentation	Displays evidence through dashboard and future GitHub checks/comments.	Reviewer-facing result

4. Technical Architecture

Architecture style

- Monorepo with independent apps and domain packages.
- FastAPI backend exposes REST APIs under /api/v1.
- Next.js dashboard consumes API contracts and displays repository/analysis evidence.
- Worker service executes long-running analysis jobs.
- PostgreSQL and SQLAlchemy support durable production storage; memory stores support local development.
- Interfaces isolate business logic from infrastructure details.

Area	Technology / package	Purpose
Frontend	Next.js, React, TypeScript	Dashboard pages, data panels, repository and analysis views
Backend API	FastAPI, Pydantic	Request validation, routing, response envelopes, API boundary
Worker	Python worker runtime	Claim analysis jobs and run the deterministic pipeline
Persistence	PostgreSQL, SQLAlchemy, Alembic	Durable repositories, analyses, graph/test/confidence artifacts
Parsing	Python AST plus JavaScript/TypeScript adapters	Source discovery, symbol/import/export extraction
Graph	Domain graph package	Dependency graph construction and reverse dependency traversal
Testing	Pytest subprocess runner and test selection package	Relevant test discovery, planning, execution result capture
GitHub	GitHub App/webhook/check/comment support	Repository events, private clone tokens, feedback publishing
Tooling	pnpm, TurboRepo, Ruff, Black, Pytest, ESLint	Workspace orchestration and code quality

5. Domain Modules

Module	Responsibility	Current implementation evidence
repository	Connected repositories, branches, pull requests, local Git ingestion.	Domain contracts, DTOs, app service, SQLAlchemy store, local clone/fetch/checkout adapter.
parser	Source file discovery and semantic extraction.	Python AST parser and JS/TS adapters for imports, exports, symbols.
graph	Repository dependency model.	Graph builder, module resolver, in-memory and SQL-backed graph stores.
analysis	Analysis lifecycle, changed files, blast radius.	Analysis service, diff analyzer, job queue, SQL stores, blast-radius computation.
testing	Discover, select, plan, and run existing tests.	Conventional discovery, path-based selector, subprocess runner, SQL artifact store.
confidence	Convert evidence into risk and confidence.	Deterministic scoring, evidence, recommendations, report persistence.
shared	Cross-cutting primitives.	Settings, database metadata, response envelopes, errors, logging.
github	GitHub App integration outputs.	Installation token flow, check-run and PR-comment publishing support.

API surface implemented for the POC

- Repository endpoints: connect, list, get, disconnect.
- Analysis endpoints: start, list, get, rerun.
- Artifact endpoints: blast radius, graph, selected tests, confidence.
- Operational endpoints: health and readiness.
- GitHub webhook endpoint with signature validation and idempotent handling.

6. Current POC and Completed Work

The current proof of concept demonstrates the deterministic foundation of Radius. It can run locally with the FastAPI API on port 8000 and the Next.js dashboard on port 3000. The backend supports memory storage for simple local review and SQL-backed storage for realistic persisted execution.

POC capability	Status
Monorepo foundation	Done - apps, packages, docs, tooling, branch workflow.
API/dashboard local setup	Done - local CORS and valid dashboard organization UUID documented.
Repository and analysis APIs	Done - route layer and service wiring exist.
Parser/graph/change/blast-radius core	Done - deterministic package implementations with tests.
Test selection and execution	Done - discovery, planning, subprocess run result capture.
Confidence reports	Done - deterministic score, risk level, evidence, recommendations.
Worker orchestration	Done - queued job claim/execute/complete/fail helpers and persisted context loading.
GitHub integration	Partially complete - webhook, token, checks/comments code exists; live credential smoke validation remains.
Docker/CI/deployment docs	Present - Dockerfiles, compose, CI workflow, deployment guide; live Docker smoke depends on local Docker/build runner.

Recent setup improvement

A local dashboard setup issue was fixed by allowing CORS requests from the dashboard origins and documenting a valid UUID for NEXT_PUBLIC_RADIUS_ORGANIZATION_ID. This prevents browser-side 'Failed to fetch' errors for new developers running the API and dashboard separately.

7. What to Demonstrate in the Introduction Round

Recommended demo narrative

#	Demo point	What to say
1	Open the dashboard	Show that Radius is not a marketing page; it is a working reviewer dashboard.
2	Explain the core question	Every page supports the question: is this pull request safe to merge?
3	Show repository and analysis concepts	Repositories create analyses; analyses own changed files, blast radius, tests, graph, and confidence.
4	Explain deterministic workflow	Show the pipeline from GitHub/manual trigger to confidence report.
5	Discuss current POC	Local API/dashboard, domain packages, worker, persistence, tests, and GitHub integration code.
6	Discuss remaining validation	Live GitHub App credentials, production-like Docker smoke, and polished final demo data.

One-minute explanation for the professor

Radius is our final year project for software verification. Modern development, especially AI-assisted development, creates code faster than teams can manually reason about risk. Our system analyzes a pull request, understands changed files and dependencies, computes the blast radius, selects relevant tests, and produces an explainable confidence report. The first version deliberately uses deterministic analysis so that every result can be traced and defended.

8. Roadmap and Next Steps

Milestone	Goal	Next action
Immediate	Complete introduction-round readiness.	Use this PDF, clean demo script, and show localhost dashboard/API.
POC validation	Validate real GitHub flow.	Configure GitHub App credentials, install app on test repository, run webhook-to-confidence demo.
Deployment smoke	Prove production runtime shape.	Run Docker Compose or CI build runner with PostgreSQL, API, worker, dashboard, migrations.
Dashboard polish	Improve review experience.	Add stable demo data, clearer empty states, analysis detail walkthrough, graph visualization improvements.
Phase 2	Repository intelligence depth.	Improve parsers, language coverage, symbol resolution, incremental parsing.
Phase 3	Dependency graph quality.	Improve graph persistence, traversal accuracy, dependency resolution, visual exploration.
Phase 4	Pull request impact analysis.	Stronger changed-symbol detection, richer blast-radius evidence.
Phase 5	Test intelligence.	Better test mapping, coverage input, framework support, faster execution.
Phase 6	Confidence engine.	Refine scoring, risk classification, evidence weighting, recommendations.
Future	AI-assisted verification.	AI explanations, generated tests, security signals, historical learning, integrations.

Important scope clarity

- Phase 1 is not trying to build all AI features.

- The current focus is the deterministic engine: repository, graph, blast radius, tests, confidence, API, worker, and dashboard.

- AI-generated tests, security analysis, historical learning, organization-wide graphs, and autonomous PR reviews are future-phase items.

9. Review Checklist and Sources

Review checklist before final project presentation

- Confirm dashboard loads from localhost and API health/readiness endpoints respond.

- Prepare one sample repository or demo analysis record so screens are not empty.

- Verify GitHub App secrets only in private environment variables, never in code.

- Run the project quality commands before final submission: pnpm install, pnpm lint, pnpm typecheck, pnpm test, pnpm build.

- If demonstrating full GitHub automation, complete the live GitHub App smoke test first.

Primary repo sources used

Source	Used for
README.md	Project identity, phase workflow, stack, structure.
docs/PROJECT_OVERVIEW.md	Problem, mission, product philosophy, target users.
docs/PHASE1_SPECIFICATION.md	Phase 1 goals, deliverables, out-of-scope items, acceptance criteria.
docs/SYSTEM_ARCHITECTURE.md	Pipeline architecture and subsystem responsibilities.
docs/API_CONTRACTS.md	API boundary, endpoint groups, response envelope.
docs/DOMAIN_MODEL.md	Repository, pull request, analysis, graph, testing, confidence concepts.
docs/PHASE1_BACKEND_TASKS.md	Completed backend work, remaining validation, current estimate.
docs/roadmap.md	Milestones and future roadmap.
docs/DEPLOYMENT.md	Services, environment, CI, GitHub App permissions.

Final takeaway: Radius is a repository-aware verification layer. Its value is not just running tests, but explaining what changed, what is affected, what was validated, and how much confidence reviewers should have before merge.